

Lustjinn

Python as a backend, for a .NET developer

A course that grows with the code. This edition was built on 5 October 2026.

Contents

0 Orientation	3
0.1 How to read this course	3
0.2 The map	3
0.3 Break 1 — Types are checked, but not enforced	4
0.4 Break 2 — There is no build step	5
0.5 Break 3 — Every project gets its own interpreter environment	5
0.6 Break 4 — Async runs on one thread	6
0.7 Smaller differences you will meet soon	6
0.8 What is already on your machine	6
0.9 What is next	7
1 Hello, deployed	8
1.1 What uv created	8
1.2 The project file	9
1.3 The app	10
1.4 How Python is served	10
1.5 The test	11
1.6 The gates	12
1.7 CI on GitHub Actions	12
1.8 The deploy	13
1.9 What step 1 leaves behind	16
2 Stories and messages	17
2.1 Settings: one typed object from the environment	17
2.2 A database in a container	18
2.3 SQLAlchemy: the engine, the session, the models	18
2.4 Alembic: the migrations	20
2.5 A rule the database enforces: messages are append-only	20
2.6 Dependency injection, FastAPI's way	21
2.7 The stories API	22
2.8 Sign-in	22
2.9 Testing against a real database	23
2.10 What step 2 leaves behind	25

CHAPTER 0

Orientation

You already know how to build a backend. You know what a request pipeline is, what dependency injection buys you, why migrations exist, how `async/await` frees a thread. None of that changes. What changes is the vocabulary, the tooling, and a handful of assumptions so deep in .NET that you no longer notice them — and those are the ones that bite.

This course follows **Lustjinn** as it is built: a web app for roleplay with a memory that does not forget, the Python successor to `airp`. Each chapter is one step of the roadmap. Claude writes the code; the chapter explains it — what each new piece is, why it is there, and what it would be in .NET. When a chapter and the code disagree, the chapter is out of date: that is a bug.

0.1 How to read this course

Every chapter has the same shape: what we are building in this step, the code that was written, the concepts it introduced, and what changed in the repository. Four kinds of box recur:

THE .NET LENS

Maps a Python idea onto the .NET one you already know — and says where the analogy breaks.

WORTH KNOWING

Background that is useful but not needed to follow the step.

WATCH OUT

A trap. Usually one that would compile — or rather, *run* — without complaint.

TRY IT

A small exercise. Optional, but the fastest way to make a concept stick.

0.2 The map

Most of the Python toolchain has a direct counterpart in what you use every day. Keep this table nearby; later chapters assume it.

You know (.NET)	Lustjinn uses (Python)	What it is
<code>dotnet CLI</code>	uv	Creates projects, installs packages, runs things
<code>nuget.org</code>	PyPI	The public package registry
<code>.csproj</code>	<code>pyproject.toml</code>	The project file: name, dependencies, tool settings
<code>packages.lock.json</code>	<code>uv.lock</code>	Exact resolved versions, committed
<code>global.json</code>	<code>.python-version</code>	Pins the runtime version
Roslyn analyzers, <code>dotnet format</code>	ruff	Linters and formatters in one
The compiler + nullable reference types	pyright (strict)	The type checker
xUnit	pytest	Tests
NSubstitute, a fake <code>HttpMessageHandler</code>	respx	Fakes HTTP calls in tests
Kestrel	uvicorn	The web server that runs the app
ASP.NET Core minimal APIs	FastAPI	Routing, model binding, DI, OpenAPI
<code>record</code> + <code>DataAnnotations</code>	Pydantic models	Typed data with validation
<code>IOptions<T></code>	pydantic-settings	Typed configuration from the environment
EF Core	SQLAlchemy 2.0	ORM and query builder
EF Core migrations	Alembic	Schema migrations
<code>HttpClient</code>	httpx	HTTP client, sync and async
<code>Task</code> , <code>async / await</code>	asyncio coroutines	Asynchronous code

The table is a starting point, not a promise. The rest of this chapter covers the four places where the mapping breaks hardest.

0.3 Break 1 — Types are checked, but not enforced

In C#, a type is a contract the compiler enforces and the runtime relies on. In Python, a type hint is an *annotation*: the interpreter stores it and otherwise ignores it.

```
def greet(name: str) -> str:
    return "Hello, " + name

greet(42)  # Runs. Fails only inside, at the "+", with a TypeError.
```

Nothing stops that call at run time. What stops it is **pyright**, run before the code ships — exactly the role the C# compiler plays, except that it is a separate tool and you could skip it. Lustjinn runs pyright in **strict** mode, in CI, on every push: that is our `TreatWarningsAsErrors`.

THE .NET LENS

`str | None` is Python's `string?`. With nullable reference types on, C# warns when you dereference something that might be null; pyright strict does the same for `None`. The difference is that C#'s `null` checks are warnings you opted into, while in strict pyright they are errors from the start.

WORTH KNOWING

Some libraries *do* read type hints at run time, on purpose. **Pydantic** and **FastAPI** are the two you will meet first: a parameter declared `limit: int` makes FastAPI parse and validate the query string as an integer and return a 422 if it is not. That is model binding, driven by the same annotations pyright checks.

0.4 Break 2 — There is no build step

`dotnet build` turns your project into an assembly; `dotnet run` runs the assembly. Python has no equivalent step you run yourself. The interpreter (**CPython**, the standard one) compiles each file to bytecode the first time it is imported, caches it in a `__pycache__` folder, and executes it. A syntax error surfaces when the file is loaded; a typo in a name surfaces only when that line runs.

That is why the tooling matters more than in .NET: **ruff** and **pyright** are the build. If they are clean and the tests pass, the code is as checked as it will get before it runs.

0.5 Break 3 — Every project gets its own interpreter environment

NuGet keeps packages in a global cache, and each project references the versions it wants; two projects can use different versions of the same package without noticing each other.

Python installs packages *into an interpreter*. Install a package into the system Python and every project on the machine sees that version. The answer is a **virtual environment**: a folder (`.venv`) holding a private copy of the interpreter's package directory, one per project.

uv manages all of it. It downloads the Python version the project pins, creates `.venv`, installs exactly what `uv.lock` says, and runs commands inside that environment:

<code>uv sync</code>	<code># make .venv match uv.lock</code>	<code>≈ dotnet restore</code>
<code>uv add httpx</code>	<code># add a dependency, update the lock</code>	<code>≈ dotnet add package</code>
<code>uv run pytest</code>	<code># run a command inside .venv</code>	<code>≈ dotnet test</code>
<code>uv python install</code>	<code># fetch the pinned Python version</code>	<code>≈ installing an SDK</code>

WATCH OUT

Your Windows machine has an older system Python (3.9) on the `PATH`. Lustjinn never uses it: `uv` fetches the version `.python-version` asks for and keeps it separate. If a command ever behaves as if it were on 3.9, it was run with `python` instead of `uv run python`.

0.6 Break 4 — Async runs on one thread

In ASP.NET Core, each request runs on a thread-pool thread. If a handler blocks — a synchronous database call, a `Thread.Sleep` — it wastes that one thread, and the pool covers for it.

Python's `asyncio` runs every coroutine on **a single thread**, inside an **event loop**. An `await` hands control back to the loop so another request can make progress. A call that blocks without awaiting does not waste one thread; it freezes *every* request the server is handling until it returns.

```
async def handler():
    time.sleep(2)           # Blocks the event loop: the whole server stops for 2 s.
    await asyncio.sleep(2)  # Yields to the loop: other requests keep running.
```

This is why Lustjinn uses the async versions of everything that waits: **httpx**'s `AsyncClient`, SQLAlchemy's async engine, an async Postgres driver.

THE .NET LENS

`async def` $\approx$ an `async Task` method, and `await` means the same thing. The difference is what happens *between* awaits: in .NET, other work runs on other threads; in Python, other work runs only when you await. There is no `ConfigureAwait`, and no `SynchronizationContext` to think about.

WORTH KNOWING

FastAPI hides one sharp edge for you. An endpoint declared with plain `def` (not `async def`) is run on a separate thread pool, so blocking inside it is safe. An endpoint declared `async def` runs on the event loop and must never block.

0.7 Smaller differences you will meet soon

- **Files are modules; folders are packages.** There is no `namespace` keyword and no project reference: `from lustjinn.stories import Story` imports by path from the package folder.
- **Naming:** `snake_case` for functions, variables and modules; `PascalCase` for classes; `UPPER_CASE` for constants. The leading underscore (`_helper`) means "private by convention" — nothing enforces it.
- **self is explicit.** A method's first parameter is the instance, written out. The constructor is `__init__`.
- **Indentation is syntax.** Blocks have no braces; the indentation *is* the block. `ruff`'s formatter keeps it consistent.
- `None`, `True`, `False` are capitalised.

0.8 What is already on your machine

The setup for this course and for step 1 is installed:

- **uv** — the Python toolchain. It will fetch the current Python on first use.
- **pandoc** and **Typst** — they turn the Markdown chapters of this course into this PDF.

The course lives in the repository under `docs/course/`. To rebuild it:

```
./docs/course/build.ps1          # the dark edition
./docs/course/build.ps1 -Light    # a light edition, for printing
```

TRY IT

Run `uv --version` and `uv python list`. The second command lists the Python versions uv can install, and any it already has. Next chapter, one of them becomes Lustjinn's.

0.9 What is next

Chapter 1 — Hello, deployed. A uv project, FastAPI with one endpoint, one pytest test, ruff and pyright in CI, and the whole thing deployed to Render — so that `GET /health` answers from the internet. It teaches the project layout, uv in practice, and how a Python web app is actually served.

CHAPTER 1

Hello, deployed

The goal of step 1 is the smallest thing that is real: a Python web app with one endpoint, one test, the quality gates, and a path to a public URL. Nothing about roleplay yet. Everything later grows out of this skeleton, so it is worth understanding every file in it.

The step landed in three pieces, and this chapter follows them in order:

1. **The project** — uv, FastAPI, one endpoint, one test, ruff and pyright.
2. **CI** — the gates running on GitHub Actions on every pull request.
3. **The deploy** — the path to Render, built and waiting: the project runs locally until it is ready for the cloud.

1.1 What uv created

One command started the project:

```
uv init --package --name lustjinn --python 3.14
```

and two more added the libraries:

```
uv add fastapi "uvicorn[standard]"
uv add --dev pytest httpx2 ruff pyright
```

The result, ignoring the docs:

<code>.python-version</code>	3.14 – which Python this project runs on
<code>pyproject.toml</code>	the project file
<code>uv.lock</code>	every package, pinned to an exact version and hash
<code>.venv/</code>	the project's private environment (gitignored)
<code>src/lustjinn/</code>	
<code>__init__.py</code>	makes the folder a package
<code>main.py</code>	the FastAPI app
<code>tests/</code>	
<code>test_health.py</code>	the one test

Python 3.14 is pinned because it is the newest stable release: 3.15 exists, but only as a release candidate. uv downloaded 3.14.7 on its own, without touching the Python 3.9 already installed on the machine.

1.1.1 Why the code lives under `src/`

The package is `src/lustjinn/`, not `lustjinn/` at the root. With the code at the root, `import lustjinn` would work in tests simply because the current folder happens to be on the import path — and a test could pass against files that would never be part of the installed app. Under `src/`, the only way to import `lustjinn` is to install it, which `uv sync` installs the project into `.venv` in *editable* mode, meaning the environment points at `src/` instead of copying it, so edits are live.

THE .NET LENS

This is the test project's `<ProjectReference>`. A test project in .NET cannot see the app's source files, only the assembly the reference builds. The `src/` layout gives Python the same discipline: tests see the package as installed, the way the server will.

1.2 The project file

`pyproject.toml` is the `.csproj`. It is TOML, a format of `[tables]` and `key = value` lines.

```
[project]
name = "lustjinn"
version = "0.1.0"
requires-python = ">=3.14"
dependencies = [
    "fastapi>=0.142.2",
    "uvicorn[standard]>=0.54.0",
]

[dependency-groups]
dev = [
    "httpx2>=2.13.1",
    "pyright>=1.1.414",
    "pytest>=9.1.1",
    "ruff>=0.16.10",
]
```

- **[project]** is the standard part every Python tool understands: name, version, the Python it needs, and the packages the app needs at run time.
- **uvicorn[standard]** — the brackets name an *extra*: optional dependencies a package offers. `standard` adds uvicorn's faster event loop and HTTP parser, and file watching for `--reload`.
- **[dependency-groups]** holds what development needs but production does not: the test runner and the gates. A production install leaves them out.
- **[build-system]** says how to turn the project into an installable package (`uv_build`).
- **[tool...]** tables configure each tool — `[tool.ruff]`, `[tool.pyright]`, `[tool.pytest...]` — all in the one file.

THE .NET LENS

`"fastapi>=0.142.2"` reads like NuGet's `Version="0.142.2"`, and both mean *at least* that version. The difference is which version gets picked: NuGet resolves the **lowest** version that satisfies every constraint, `uv` the **highest**. Either way, the lock file is what makes the build repeatable — `uv.lock` plays the role of `packages.lock.json`, and it is committed.

1.3 The app

The first version of the whole application, `src/lustjinn/main.py` (the deploy, at the end of the chapter, grows it by one field):

```
"""The FastAPI application: the object uvicorn serves."""

from fastapi import FastAPI

app = FastAPI(title="Lustjinn")

@app.get("/health")
async def health() -> dict[str, str]:
    """Answers while the process is up. Render and the PWA's waking screen poll it."""
    return {"status": "ok"}
```

Line by line:

- The string at the top of a file, function or class is a **docstring**: documentation the language keeps (`help()` shows it). It plays the part of an XML `<summary>` comment.
- `from fastapi import FastAPI` imports one name from a package — a `using` that names exactly what it brings in.
- `app = FastAPI(...)` creates the application at module level. When Python imports this file, the line runs, and `app` exists from then on.
- `@app.get("/health")` is a **decorator**. Python calls `app.get("/health")`, which returns a function; that function receives `health`, registers it as the handler for `GET /health`, and hands it back unchanged. The `@` line is shorthand for exactly that.
- `async def health() -> dict[str, str]` declares a coroutine that returns a dictionary of strings. FastAPI reads the return annotation: it documents the response in the OpenAPI schema and validates what the function returns.
- `return {"status": "ok"}` — FastAPI serialises the dictionary to JSON.

THE .NET LENS

Put side by side with a minimal API, there is almost nothing to translate:

```
var app = WebApplication.CreateBuilder(args).Build();
app.MapGet("/health", () => new Dictionary<string, string> { ["status"] = "ok" });
app.Run();
```

`app.MapGet(...)` becomes the decorator, and `Dictionary<string, string>` becomes `dict[str, str]`. What has no counterpart is `app.Run()` — see the next section.

1.4 How Python is served

A .NET app is an executable: `Program.cs` builds the app and `app.Run()` starts Kestrel inside the same process. A Python web app is not run; it is **imported by a server**. The server here is **uvicorn**:

```
uv run uvicorn lustjinn.main:app --reload
```

`lustjinn.main:app` means *import the module `lustjinn.main` and serve the object called `app` in it*. `uvicorn` owns the process, the socket and the event loop; FastAPI only answers requests handed to it. `--reload` restarts the server when a file changes — for development only.

The contract between the two is **ASGI** (Asynchronous Server Gateway Interface): a standard signature every async Python server can call and every async framework implements. That is why FastAPI could run on another ASGI server without changing a line.

THE .NET LENS

`uvicorn` is Kestrel, and ASGI is the boundary between Kestrel and the ASP.NET Core pipeline — except that in Python the boundary is a public standard, so servers and frameworks mix and match.

With the server running, FastAPI also serves its own documentation: `/docs` is an interactive Swagger UI and `/openapi.json` the schema, both generated from the code — no Swashbuckle to add.

1.5 The test

The first version of `tests/test_health.py`:

```
from fastapi.testclient import TestClient

from lustjinn.main import app

client = TestClient(app)

def test_health_answers_ok() -> None:
    response = client.get("/health")

    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

- **pytest finds tests by name:** files called `test_*.py`, functions called `test_*`. No attribute, no test class needed.
- **Plain `assert`.** `pytest` rewrites `assert` statements when it loads a test file, so a failure shows both sides of the comparison — no `Assert.Equal` family to learn.
- **`TestClient`** runs the app in-process and speaks to it over ASGI directly: no server, no port, no network.

THE .NET LENS

`TestClient(app)` is `WebApplicationFactory<Program>` plus `CreateClient()`, in one line.

WATCH OUT

When this test was first written, the dev dependency was `httpx`, as countless tutorials say. The test passed — with a deprecation warning, because `Starlette` (the toolkit under FastAPI) now wants its successor, `httpx2`. `pyright` went further and failed: with `httpx2` missing, it could not know the types of `client.get`, `response.status_code` or `response.json`, and strict mode refuses unknown types. Two lessons in one. Tutorials age fast in Python — check what the

library itself asks for. And strict typing earns its keep on day one: it turned a quiet warning into an error.

1.6 The gates

Four commands decide whether a change is acceptable. All four are clean on this commit:

```
uv run pytest          # the tests
uv run ruff check      # lint
uv run ruff format --check # formatting, without rewriting
uv run pyright         # types, strict
```

ruff is configured in `pyproject.toml` with a deliberate set of rule families. Most are style and likely-bug checks. One deserves a mention: **ASYNC** flags blocking calls made inside `async def` functions — the event-loop freeze from chapter 0, caught before it runs.

pyright runs in `strict` mode over `src` and `tests`. Strict means every value must have a type pyright can know; an unknown type is an error, not a shrug.

TRY IT

Start the server with `uv run uvicorn lustjinn.main:app --reload` and open `http://127.0.0.1:8000/docs`. Call `/health` from the Swagger page.

Then, in `main.py`, change the return to `{"status": 1}` and run `uv run pyright`. The annotation says `dict[str, str]`; pyright says no. Change it back.

1.7 CI on GitHub Actions

The gates are only worth something if nothing reaches `develop` without passing them. A workflow in `.github/workflows/ci.yml` runs all four on GitHub, where the repository lives:

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main, develop]

concurrency:
  group: ci-${{ github.event.pull_request.number || github.run_id }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  gates:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    env:
      UV_VERSION: "0.12.22"
    steps:
      - uses: actions/checkout@v5
        with:
          persist-credentials: false
      - name: Install uv
```

```

run: |
  curl -LsSf "https://astral.sh/uv/${UV_VERSION}/install.sh" | sh
  echo "$HOME/.local/bin" >> "$GITHUB_PATH"
- name: Sync the environment from uv.lock
  run: uv sync --locked
- name: ruff check
  run: uv run ruff check
- name: ruff format
  run: uv run ruff format --check
- name: pyright (strict)
  run: uv run pyright
- name: pytest
  run: uv run pytest

```

Reading it top to bottom:

- **on** — a pull request into `develop` or `main` runs the gates. A push to `develop` does **not**: `develop` only moves by merging a pull request whose gates already ran, so running them again would test the same code twice. A push to `main` — a release — does.
- **concurrency** — a new push to a pull request cancels that pull request's previous run. Only the newest commit's result matters.
- **runs-on: ubuntu-latest** — a fresh Linux virtual machine for every run. `uv` and the project's Python come from the steps below, so a green run proves the project builds from its committed files alone.
- **timeout-minutes** — a stuck job stops after ten minutes. The default is six hours.
- **Install uv** — one pinned release, the same version as on the development machine. Python then comes from `uv`, exactly as on Windows: `uv sync` reads `.python-version` and fetches 3.14.
- **uv sync --locked** — creates the environment from `uv.lock`, and **fails** if the lock no longer matches `pyproject.toml`. Without `--locked`, `uv` would quietly re-resolve, and CI would test versions nobody committed.
- **The gates, cheapest first** — a formatting slip fails in seconds, before `pyright` and the tests run.

THE .NET LENS

`uv sync --locked` is `dotnet restore --locked-mode`: restore exactly what the lock file says, or fail. And the workflow as a whole is the pipeline you would write around `dotnet build` and `dotnet test` — with `uv sync` as the restore and `pyright` and `ruff` standing in for the build.

WORTH KNOWING

The repository is private, so every minute a workflow runs is billed against the account's monthly allowance, and each job is rounded up to a whole minute. That is the reason for three of the choices above: nothing runs on a `develop` push, superseded runs are cancelled, and every job has a timeout. A run of these gates takes under a minute.

1.8 The deploy

The last piece is the path to the internet: a **Render** free web service, called `lustjinn-staging`, serving the `develop` branch. The path is built and tested as far as it can be without an account — but it has not run yet. The project stays on the development machine until it is worth putting in the cloud, and this section describes what will happen then.

1.8.1 What Render runs

Render's native Python runtime needs no Dockerfile. It reads `.python-version` to choose the interpreter, and because the repository contains `uv.lock`, it installs `uv` too. The service is described in `render.yaml` at the root of the repository — a **blueprint**:

```
services:
  - type: web
    name: lustjinn-staging
    runtime: python
    plan: free
    branch: develop
    buildCommand: uv sync --locked --no-dev
    startCommand: uv run --no-sync uvicorn lustjinn.main:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /health
    autoDeployTrigger: "off"
    envVars:
      - key: UV_VERSION
        value: "0.12.22"
```

- **The build** is the same `uv sync --locked` as CI, plus `--no-dev`: `pytest`, `ruff` and `pyright` stay out of production.
- **The start** runs `uvicorn` through `uv run --no-sync` — inside the environment the build created, without checking it again. It binds to `0.0.0.0`, every network interface, because Render's proxy reaches the process from outside the container; the default, `127.0.0.1`, would accept only connections from inside. And it listens on `$PORT`, the port Render chooses.
- **healthCheckPath** — Render calls `/health` to decide whether the service is up.
- **autoDeployTrigger: "off"** — a push alone deploys nothing; a deploy is always asked for.

THE .NET LENS

`--host 0.0.0.0 --port $PORT` is Kestrel's `ASPNETCORE_URLS=http://+:$PORT` — the same lesson every container platform teaches: listen where the platform says, on every interface. And the blueprint is infrastructure as code, a very small Bicep file: the service lives in the repository, not only in a dashboard.

WATCH OUT

The free tier has two properties the whole design respects. The service **sleeps** after about fifteen minutes without requests and takes 30–60 seconds to wake — which is why the future PWA is a separate static site that opens instantly and shows “waking the server”. And it has **no persistent disk**: anything written to the filesystem disappears on the next deploy or restart. No SQLite, no files. Everything lives in the database: a local Postgres for now, Neon once the app moves to the cloud.

1.8.2 How a deploy travels

A deploy is started by hand: on GitHub, *Actions* → *CI* → *Run workflow* on `develop`, with **deploy = staging**. The same workflow that runs the gates then runs one more job, `deploy-staging`, only if they all passed:

1. **Fire the hook.** A `POST` to Render's *deploy hook* — a secret URL — starts a build of the branch.

2. **Wait for the right commit.** The job polls `/health` until the service reports *this* commit. “The service is up” is not enough: the old version is up too, until the new one replaces it.

The one-time setup — creating the service from the blueprint, the hook secret and the service URL on GitHub — is in `docs/DEPLOYMENT.md`.

1.8.3 `/health` grows a field

Step 2 of that list needs the service to say which commit it runs. Render provides it in an environment variable, `RENDER_GIT_COMMIT`, so `/health` now returns it:

```
import os
from typing import Literal

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Lustjinn")

class Health(BaseModel):
    status: Literal["ok"]
    commit: str | None
    """The deployed commit (Render's `RENDER_GIT_COMMIT`); None when not running on Render."""

@app.get("/health")
async def health() -> Health:
    """Answers while the process is up. Render, the deploy job and the PWA poll it."""
    return Health(status="ok", commit=os.environ.get("RENDER_GIT_COMMIT"))
```

This is the first **Pydantic model**. A class that inherits `BaseModel` declares its fields as annotated class attributes; Pydantic validates them when an instance is created and serialises them to JSON. FastAPI shows the model in the OpenAPI schema under its own name, `Health`.

- `Literal["ok"]` — a type with exactly one allowed value. `Health(status="fine")` fails validation, and pyright rejects it before it runs.
- `str | None` — a string or nothing. `os.environ.get(...)` returns `None` when the variable is not set, so off Render the answer is `{"status": "ok", "commit": null}`.

THE .NET LENS

`Health` is a record `Health(string Status, string? Commit)` with validation attached. `os.environ.get` is `Environment.GetEnvironmentVariable`, which also returns null when the variable is missing. `Literal["ok"]` has no direct C# counterpart; the nearest is a one-value enum.

The tests grow with it:

```
def test_health_reports_the_deployed_commit(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.setenv("RENDER_GIT_COMMIT", "3d36977")

    assert client.get("/health").json()["commit"] == "3d36977"

def test_health_reports_no_commit_off_render(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.delenv("RENDER_GIT_COMMIT", raising=False)
```

```
assert client.get("/health").json() == {"status": "ok", "commit": None}
```

`monkeypatch` is a **fixture**: pytest sees a parameter with that name and passes in a ready-made object. `setenv` and `delenv` change the environment for this test only — pytest undoes both when it ends, so no test leaks state into the next.

THE .NET LENS

Fixtures are pytest's dependency injection — by parameter name, the way ASP.NET Core injects by type. xUnit's nearest equivalents are constructor injection and `IClassFixture<T>`; the automatic undo is what you would otherwise write in `Dispose()`.

TRY IT

Run the server with a commit of your own, in PowerShell:

```
$env:RENDER_GIT_COMMIT = "hello"; uv run uvicorn lustjinn.main:app
```

`/health` now reports `"commit": "hello"`, and `/docs` shows the `Health` schema. Then try `Health(status="fine", commit=None)` in `main.py` and run `uv run pyright`.

1.9 What step 1 leaves behind

- A Python 3.14 project managed by `uv`, locked, with a `src/` layout.
- A FastAPI app with one endpoint, three tests, and four gates — all clean.
- CI on GitHub Actions running the gates on every pull request.
- A deploy path to Render staging that refuses to run on red gates and checks what it deployed — ready for the day the app moves to the cloud.

Next — Chapter 2, Stories and messages. The first real data, in a Postgres running in Docker: settings from the environment, sign-in, SQLAlchemy models, the first Alembic migration, and a trigger that makes messages append-only.

CHAPTER 2

Stories and messages

Step 1 built an app that could say it was alive. Step 2 gives it something to keep: a database, the tables for stories and their messages, a rule the database enforces by itself, an API over them, and a lock on the door. Everything runs on your machine — Postgres in a Docker container, the API under uvicorn. The cloud comes later and changes one connection string.

This is the chapter where Python's backend stack stops looking like a toy and starts looking like what you already know. Almost every piece has a .NET twin; the interesting parts are the few places where the twin behaves differently.

2.1 Settings: one typed object from the environment

Before anything can connect to anything, the app needs to be told where the database is and who may sign in. `src/lustjinn/settings.py`:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="LUSTJINN_", env_file=".env", extra="ignore")

    database_url: str
    username: str
    password: SecretStr
    token_secret: SecretStr = Field(min_length=32)
    token_days: int = Field(default=30, ge=1, le=365)
    cors_origins: list[str] = []

    @lru_cache
    def get_settings() -> Settings:
        return Settings()
```

`BaseSettings` is a Pydantic model that fills itself in: each field is read from the environment variable of the same name with the prefix — `LUSTJINN_DATABASE_URL`, `LUSTJINN_USERNAME` — or from a `.env` file in the working directory, real variables winning. A field with no default is required. Types are enforced: `token_days` must parse as an integer between 1 and 365.

- **SecretStr** wraps a string so that printing or logging the settings shows `*****`. You get the value out on purpose, with `.get_secret_value()`.
- **@lru_cache** on a function with no arguments turns it into “compute once, then return the same object”: the settings are read on the first call and reused after that.

- **Failing early.** The app's *lifespan* — code FastAPI runs once at start-up and once at shutdown — calls `get_settings()`. A missing password stops uvicorn before it accepts a single request, with an error naming the field.

THE .NET LENS

`Settings` is the class you would bind with `services.Configure<Settings>(config)` and receive as `IOptions<Settings>`; the environment-variable and `.env` layering is the configuration builder; `Field(min_length=32)` is a `DataAnnotation` plus `ValidateOnStart()`. The lifespan is `IHostedService.StartAsync / StopAsync` in one function, split by a `yield`.

`.env` is gitignored. `.env.example` is committed and lists every variable; copy it and fill in the blanks.

2.2 A database in a container

`docker-compose.yml` describes one service: Postgres 17 with the `pgvector` extension available, listening on port 5440 of this machine only, its data in a named volume.

```
docker compose up -d      # start it; the data survives restarts
docker compose down      # stop it; add -v to throw the data away
```

Port 5440 rather than the usual 5432 so it never collides with another project's Postgres. Postgres 17 because Neon offers it: when the app moves to the cloud, the schema and every query stay as they are.

2.3 SQLAlchemy: the engine, the session, the models

SQLAlchemy is the ORM. Three objects carry it, and each has an EF Core counterpart — though the unit of work is more explicit here.

2.3.1 The engine and the session

```
@lru_cache
def get_engine() -> AsyncEngine:
    return create_async_engine(get_settings().database_url, pool_pre_ping=True)

@lru_cache
def get_sessionmaker() -> async_sessionmaker[AsyncSession]:
    return async_sessionmaker(get_engine(), expire_on_commit=False)

async def get_session() -> AsyncGenerator[AsyncSession]:
    async with get_sessionmaker()() as session:
        yield session
```

- The **engine** owns the connection pool. One per process, created on first use.
- A **session** is one conversation with the database: it tracks the objects you loaded or added, and writes the changes when told. One per request.
- **get_session** is a generator: it yields a session to whoever asked, and the code after the `yield` — here, leaving the `async with`, which closes the session — runs when the request ends.

THE .NET LENS

The engine is the pooled connection factory behind `AddDbContext`. The session is the `DbContext`: change tracking, identity map, one unit of work, scoped to a request. Two differences will matter all through this project:

- **flush() and commit() are separate.** `flush()` sends the pending INSERTs and UPDATEs but leaves the transaction open; `commit()` ends it. EF's `SaveChanges()` does both.
- **There is no lazy loading in async code.** Touching `story.messages` cannot quietly run a query, because a query needs an `await`. The relationships are declared `lazy="raise"`, so a forgotten load fails loudly instead of deadlocking or lying. You load what you need, in the query — the discipline EF asks for with `Include`, enforced.

`expire_on_commit=False` is the other async accommodation: by default SQLAlchemy forgets an object's values at commit and reloads them on next access, which again would be a hidden query.

2.3.2 The models

`src/lustjinn/models.py`, abridged:

```
class Story(Base):
    __tablename__ = "stories"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=new_id)
    name: Mapped[str] = mapped_column(String(200))
    character_id: Mapped[uuid.UUID] = mapped_column(
        ForeignKey("characters.id", ondelete="RESTRICT"), index=True
    )
    persona_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("personas.id", ondelete="RESTRICT"), index=True
    )
    model: Mapped[str | None] = mapped_column(String(200))
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    deleted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
```

A model is a class whose annotated attributes are columns. `Mapped[str]` is a NOT NULL text column; `Mapped[str | None]` is nullable — the type hint is the nullability, checked by pyright and enforced by Postgres. `mapped_column(...)` adds what the type cannot say: lengths, keys, defaults, indexes.

Four tables exist after this step: `characters` and `personas` (minimal for now; the library step fills them out), `stories`, and `messages`. The decisions worth noticing:

- **Ids are UUID version 7** (`uuid.uuid7()`, new in Python 3.14): unique like any UUID, but beginning with a timestamp, so they sort by creation time and index well.
- **A story points at its character by id**, with `ON DELETE RESTRICT`. `airp` stored the character's *name*, so renaming one was impossible, and "not while a story uses it" was checked in code. Here the database refuses the delete itself.
- **Deleting is hiding**. `deleted_at` is set and every query filters on it. The row stays.
- **A message has a sequence**, unique within its story, and a `request_hash` that is unique within its story *among rows that have one* — a **partial index**. Step 3 uses the hash to make a retried request harmless.
- **model is null when a person wrote the message**. The reader's turns, and a story's opening. A later step reads that null to know an opening must not be rerolled.

THE .NET LENS

`Mapped[...]` and `mapped_column` are the entity class and its `OnModelCreating` configuration merged into one declaration. `ForeignKey(..., onDelete="RESTRICT")` is `.onDelete(DeleteBehavior.Restrict)`. The partial index is `HasIndex(...).HasFilter("...")`.

2.4 Alembic: the migrations

The models say what the schema should be. **Alembic** gets a database there, one versioned step at a time, and records in a table (`alembic_version`) how far it has gone.

```
uv run alembic revision --autogenerate -m "what changes" # write a migration from the models
uv run alembic upgrade head                             # apply what is not applied yet
uv run alembic downgrade -1                             # undo the last one
```

Autogenerate compares the models with a live database and writes the difference as Python: `op.create_table(...)`, `op.create_index(...)`. The result is a draft — it is read, tidied and committed like any code. `migrations/versions/0001_stories_and_messages.py` began that way. `migrations/env.py` is the file Alembic runs for every command. Ours does two things: it takes the database URL from `get_settings()`, so there is one source of truth, and it accepts a connection handed in from outside — which is how the tests run the real migrations against their own database.

THE .NET LENS

`alembic revision --autogenerate` is `dotnet ef migrations add`; `alembic upgrade head` is `dotnet ef database update`; `alembic_version` is `__EFMigrationsHistory`. The difference: there is no model snapshot file. Alembic compares against a real database every time, so the database must be at the previous migration when you generate the next.

One test keeps the two descriptions honest: it applies every migration to an empty database and asks Alembic whether anything still differs from the models. A model edited without a migration fails the build.

2.5 A rule the database enforces: messages are append-only

A story's memory is derived from its transcript — summaries, facts, embeddings. If a message could be edited or deleted, everything built from it would silently become a lie. `airp` guarded this in `SaveChanges`. That protects against the application; it does nothing about a script, a console session, or a second client. So here the rule lives where the data lives.

Migration `0002` is hand-written SQL, because a trigger is not something a model can express:

```
CREATE FUNCTION messages_append_only() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        IF current_setting('lustjinn.purging', true) = 'on' THEN
            RETURN OLD;
        END IF;
        RAISE EXCEPTION 'messages are append-only: a message is hidden with deleted_at, ...'
```

```

        USING ERRCODE = 'restrict_violation';
    END IF;

    IF NEW.text IS DISTINCT FROM OLD.text
    OR NEW.role IS DISTINCT FROM OLD.role
    OR NEW.story_id IS DISTINCT FROM OLD.story_id
    OR NEW.sequence IS DISTINCT FROM OLD.sequence THEN
        RAISE EXCEPTION 'messages are append-only: text, role, story and sequence cannot ...'
        USING ERRCODE = 'restrict_violation';
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER messages_append_only
BEFORE UPDATE OR DELETE ON messages
FOR EACH ROW EXECUTE FUNCTION messages_append_only();

```

A trigger function runs for each affected row, before the change. `OLD` and `NEW` are the row before and after; `RAISE EXCEPTION` aborts the statement. Reading it:

- **A delete is refused** — unless the transaction has announced a purge with `SET LOCAL lustjinn.purging = 'on'`. That is the one door, for the day a story is erased for good, and it closes by itself when the transaction ends.
- **What a turn said, who said it and where it sits cannot change** — purge or no purge.
- **Everything else may**: `deleted_at`, to hide a message; `request_hash`; columns later steps add.
- A second trigger refuses `TRUNCATE`, which empties a table without visiting rows and so never fires the first.

The error code is one of Postgres's own integrity codes, so SQLAlchemy raises `IntegrityError` — the same exception as for a broken foreign key. To the application the trigger is one more constraint.

WORTH KNOWING

`IS DISTINCT FROM` is `<>` that treats `NULL` as a value: `NULL <> NULL` is unknown, while `NULL IS DISTINCT FROM NULL` is false. In a guard you want the second.

2.6 Dependency injection, FastAPI's way

Every endpoint that touches the database needs a session. It asks for one in its signature:

```

Session = Annotated[AsyncSession, Depends(get_session)]

@router.get("")
async def list_stories(session: Session) -> list[StoryOut]:
    rows = await session.execute(_stories())
    return [_out(*row) for row in rows]

```

`Depends(get_session)` tells FastAPI: before calling this function, call `get_session` and pass in what it yields. `Annotated[X, ...]` attaches that instruction to the type, and the alias `Session` gives the pair a name so every endpoint can write `session: Session`.

There is no container and no registration step. A dependency is any callable; its own parameters can be dependencies in turn; a generator dependency gets its cleanup run after the response.

THE .NET LENS

`Depends(get_session)` is constructor injection of a scoped service, declared at the point of use instead of in `Program.cs`. The lifetime is per request, like `AddScoped`. What replaces `ConfigureTestServices` is `app.dependency_overrides[get_session] = something_else` — a dictionary from the real provider to its stand-in. The tests use exactly that.

2.7 The stories API

`src/lustjinn/stories.py` holds five endpoints under `/stories`: create, list, read one with its messages, rename, delete. Three things in it are new.

Pydantic models are the DTOs. What comes in and what goes out are declared as classes, and FastAPI validates the first and serialises the second:

```
Name = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=200)]

class NewStory(BaseModel):
    name: Name
    character_id: uuid.UUID
    persona_id: uuid.UUID | None = None
    model: ModelName | None = None
```

A body with a blank name, or an id that is not a UUID, never reaches the function: FastAPI answers 422 with the field and the reason.

A router groups endpoints. `APIRouter(prefix="/stories")` is `MapGroup("/stories")`; the app includes it with `app.include_router(...)`.

The list is one query. For each story it needs the newest visible message — its time, to sort by, and its first 200 characters, to show under the name. That is a **LATERAL** subquery: one that runs per outer row and may refer to it. EF Core produces the same SQL from a correlated `OrderByDescending(...).FirstOrDefault()`; here it is written out.

Creating a story does one more thing: if its character has an **opening**, the opening becomes message 1, role `assistant`, with no model — a page a person wrote, not a reply to regenerate.

2.8 Sign-in

The API will one day sit on a public URL with one user's private stories behind it.

`src/lustjinn/auth.py`:

1. `POST /auth/sign-in` takes the username and password and compares them with the settings.
2. If they match, it returns a **token**: a small signed document saying who it is for and when it expires (30 days by default).
3. Every other request carries `Authorization: Bearer <token>`. A dependency, `require_user`, checks the signature and the expiry before the endpoint runs.

The token is a **JWT** signed with HMAC-SHA256 using `LUSTJINN_TOKEN_SECRET`. Nothing is stored on the server: the signature proves the token was issued here, and the expiry is inside it. Changing the secret invalidates every token at once — that is “sign out everywhere”.

The guard is attached where the router is included, not on each endpoint:

```
app.include_router(auth.router)
# Everything below needs a valid token.
app.include_router(stories.router, dependencies=[Depends(auth.require_user)])
```

THE .NET LENS

This is `AddAuthentication().AddJwtBearer()` plus `MapGroup("/stories").RequireAuthorization()`, without the middleware: the check is an ordinary dependency. `HTTPBearer` is the piece that reads the header and puts an *Authorize* button on the `/docs` page.

Details that are easy to get wrong, each pinned by a test:

- **Comparison in constant time** (`secrets.compare_digest`), and both name and password are checked before answering, so neither the reply nor its timing says which one was wrong.
- **The accepted algorithm is named explicitly**. A forged token whose header says “no signature needed” is refused because the server never asks the token how to verify it.
- **A token without an expiry is refused**, as is one issued for a different username.
- **One test covers every route**: it reads the app’s own OpenAPI description and requires a 401 from each path except `/health` and `/auth/sign-in`. An endpoint added next month is guarded, or that test fails.

WATCH OUT

FastAPI’s default 422 response includes the input it rejected. For a malformed sign-in that input is the password; for a story it is the text. A test caught the first, and the app now answers validation errors with the field and the problem only. Defaults are written for tutorials — read what yours actually send.

Two more things ride along. **CORS** — which browser origins may call the API — comes from the settings and allows none by default; the web client’s origin gets added when it exists. And every response carries four headers saying it is private: not to be cached, indexed, or referred from.

2.9 Testing against a real database

The trigger, the constraints and the queries are Postgres behaviour. Faking the database would test the fake. So the tests use the real thing — and never the data you are playing with.

`tests/conftest.py` builds it from **fixtures**, each asking for the one before by naming it as a parameter:

1. **database_url** (once per run) creates an empty database with a random name on the same server, and drops it at the end.
2. **engine** (once per run) applies every Alembic migration to it — the real migrations, so they are tested on every run.

3. **session** (per test) opens a transaction that is **never committed**, and rolls it back when the test ends. The code under test may call `commit()` freely: the session turns those into savepoints inside the outer transaction.
4. **client** (per test) is an HTTP client wired straight to the app in-process, with the app's `get_session` and `get_settings` overridden to hand out the test's own.

```
async def test_deleting_a_story_hides_it_and_keeps_its_rows(
    client: httpx2.AsyncClient, session: AsyncSession
) -> None:
    story = await a_story(session)
    await session.commit()

    response = await client.delete(f"/stories/{story.id}")

    assert response.status_code == 204
    assert (await client.get(f"/stories/{story.id}")).status_code == 404
    kept = await session.get(Story, story.id, populate_existing=True)
    assert kept is not None and kept.deleted_at is not None
```

The test arranges through the session, acts through HTTP, and checks both the response and the row — all inside one transaction that leaves nothing behind.

THE .NET LENS

`scope="session"` fixtures are `IClassFixture` /collection fixtures; the per-test rollback is what `Respawn` or a transaction scope gives you in .NET; the in-process client is `WebApplicationFactory.CreateClient()`. `async def` tests need a plugin, `pytest-asyncio`, configured here to run everything on one event loop — the engine's connections belong to it.

Tests that need a character use the **dummy**: "The Gilded Heron", a fictional card of about 1,850 words with an opening and a persona, under `tests/fixtures/dummy/`. `airp`'s tests used a four-word card inline and missed a bug that cost a real story twenty-four turns of memory; from step 5 on, the memory tests here run on a card the size of a real one.

In CI the same Postgres image runs as a *service container* beside the job, on the same port, so the tests' default connection string works there unchanged.

TRY IT

Bring it all up and use it:

```
cp .env.example .env # fill in a username, a password, a token secret
docker compose up -d
uv run alembic upgrade head
uv run python scripts/seed_dummy.py # prints the dummy character's id
uv run uvicorn lustjinn.main:app --reload
```

Open `http://127.0.0.1:8000/docs`. Call `POST /auth/sign-in`, copy the `access_token`, press **Authorize** and paste it. Then `POST /stories` with a name and the character id the seed script printed — the response's first message is the opening.

Now try to break the rule. In another terminal:

```
docker compose exec db psql -U lustjinn -c "UPDATE messages SET text = 'rewritten'"
docker compose exec db psql -U lustjinn -c "DELETE FROM messages"
```

Both are refused, by the database, with no application running in between.

2.10 What step 2 leaves behind

- Typed settings that stop the app at start-up when something is missing.
- A local Postgres, four tables, two migrations, and a test that keeps models and migrations in step.
- Messages that cannot be rewritten or deleted, whoever asks.
- A stories API behind a bearer token, with every route's guard tested from the app's own schema.
- A test suite that runs against real Postgres and leaves no trace.

Next — Chapter 3, One turn against OpenRouter. The first conversation with a model: an async HTTP client, the reply streamed to the caller as it is written, the reader's turn saved before the model is asked, and a ledger row for what it cost.